

Fundamentals of Information Systems Security

Lab 8: Advanced Password Cracking

Understanding Password Security and Attack Techniques

Version: 1.0**Estimated Time:** 2-3 Hours**Difficulty Level:** Advanced**Prerequisites:** Labs 1, 3

Contents

1	Introduction	2
1.1	The Importance of Password Security	2
1.2	Learning Objectives	2
1.3	Prerequisites	2
1.4	Important Legal Notice	2
2	Understanding Password Storage	3
2.1	Common Hashing Algorithms	3
2.2	Salting and Key Stretching	3
3	Password Cracking Techniques	3
3.1	Cracking Methodology	3
3.2	Attack Comparison	3
4	Lab Exercises	5
4.1	Exercise 1: Understanding Password Hashes	5
4.2	Exercise 2: Dictionary Attacks with Hashcat	5
4.3	Exercise 3: Rule-Based Attacks	6
4.4	Exercise 4: Mask Attacks	6
4.5	Exercise 5: GPU Acceleration	6
4.6	Exercise 6: Rainbow Tables	7
5	Defensive Measures	8
6	Deliverables	8
7	Additional Resources	8

1 Introduction

1.1 The Importance of Password Security

Passwords remain the most common authentication mechanism despite known weaknesses. Password cracking is the process of recovering passwords from stored data or transmitted authentication tokens. Understanding password cracking techniques is essential for both attackers and defenders.

Why Password Cracking Matters

- Passwords are the primary authentication method for most systems
- Weak passwords are easily cracked
- Password reuse increases attack surface
- Stolen credentials enable further attacks
- Demonstrates importance of password policies

1.2 Learning Objectives

By the end of this lab, you will be able to:

1. Understand different password hashing algorithms
2. Perform dictionary-based password attacks
3. Use rule-based attacks for password variation
4. Apply mask attacks for brute-force optimization
5. Use GPU acceleration for faster cracking
6. Understand rainbow tables and their limitations
7. Implement effective password policies and defenses

1.3 Prerequisites

- Kali Linux VM (from Lab 1)
- Basic understanding of hashing algorithms (from Chapter 6)
- Familiarity with John the Ripper (from Lab 3)
- Understanding of password security concepts

1.4 Important Legal Notice

Important Legal Notice

- Password cracking should only be performed on systems you **own** or have **explicit permission** to test
- Unauthorized access to computer systems is illegal
- Use these skills to improve security, not to harm others

2 Understanding Password Storage

2.1 Common Hashing Algorithms

Algorithm	Status	Characteristics
MD5	Deprecated	128-bit, fast, collision vulnerabilities
SHA-1	Deprecated	160-bit, collision vulnerabilities
SHA-256	Secure	256-bit, widely used
SHA-512	Secure	512-bit, more secure than SHA-256
bcrypt	Secure	Adaptive, salt included, slower
scrypt	Secure	Memory-hard, resistant to GPU attacks
Argon2	Most Secure	Memory-hard, winner of Password Hashing Competition
NTLM	Insecure	Windows, vulnerable to pass-the-hash

2.2 Salting and Key Stretching

Security Mechanisms

- **Salting:** Adding random data to each password before hashing
- **Peppering:** Adding a secret key to the password before hashing
- **Key Stretching:** Repeatedly hashing to increase computation time
- **PBKDF2:** Password-Based Key Derivation Function 2
- **bcrypt:** Adaptive hash function that slows down as hardware improves
- **scrypt:** Memory-hard function resistant to GPU attacks
- **Argon2:** Modern, memory-hard function (winner of PHC)

3 Password Cracking Techniques

3.1 Cracking Methodology

Attack Categories

1. **Brute Force:** Try every possible combination
2. **Dictionary Attack:** Use a list of common passwords
3. **Hybrid Attack:** Dictionary + variations
4. **Rule-Based Attack:** Apply transformation rules
5. **Mask Attack:** Use patterns (known structure)
6. **Rainbow Table:** Precomputed hash chains

3.2 Attack Comparison

highlightblue Method	Speed	Success Rate	Use Case
Dictionary	Fast	High (weak passwords)	Initial attempt
Rule-Based	Fast	High (common patterns)	Password variations
Mask Attack	Medium	Medium	Known password format
Brute Force	Slow	Low (complex)	Last resort
Rainbow Table	Very Fast	Medium (precomputed)	Common passwords

4 Lab Exercises

4.1 Exercise 1: Understanding Password Hashes

Exercise 1: Hash Generation

Objective: Generate and understand different password hashes.

1. Generate MD5 hash:

```
echo -n "password123" | md5sum  
# Output: 482c811da5d5b4bc6d497ffa98491e38
```

2. Generate SHA-256 hash:

```
echo -n "password123" | sha256sum  
# Output:  
ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
```

3. Generate SHA-512 hash:

```
echo -n "password123" | sha512sum
```

4. Generate a salted hash:

```
mkpasswd -m sha-512 -S 12345678 password123
```

5. **Reflection:** Why is salting important for password security?

4.2 Exercise 2: Dictionary Attacks with Hashcat

Exercise 2: Dictionary Attacks

Objective: Use advanced wordlists and dictionary attacks with Hashcat.

1. Install Hashcat:

```
sudo apt-get update  
sudo apt-get install hashcat
```

2. Extract rockyou.txt:

```
sudo gunzip /usr/share/wordlists/rockyou.txt.gz
```

3. Create hashes.txt with these MD5 hashes:

```
# password -> 5f4dcc3b5aa765d61d8327deb882cf99  
# 123456 -> e10adc3949ba59abbe56e057f20f883e  
# qwerty -> d8578edf8458ce06fbc5bb76a58c5ca4
```

4. Run Hashcat:

```
hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
```

5. **Reflection:** What are the limitations of dictionary attacks?

4.3 Exercise 3: Rule-Based Attacks

Exercise 3: Rule-Based Attacks

Objective: Use rules to expand dictionary attacks.

1. Use John the Ripper with rules:

```
john --rules --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
```

2. Use Hashcat with rules:

```
hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule
```

3. Create a custom rule:

```
echo 'c $1 $!' > custom.rule  
# Capitalize + append 1 + append !
```

4. **Reflection:** How do rules improve password cracking success?

4.4 Exercise 4: Mask Attacks

Exercise 4: Mask Attacks

Objective: Use mask attacks for targeted password cracking.

1. Mask syntax:

```
?l = Lowercase letters  
?u = Uppercase letters  
?d = Digits  
?s = Special characters  
?a = All characters
```

2. Run mask attacks:

```
# 8-digit PIN  
hashcat -m 0 -a 3 hashes.txt ?d?d?d?d?d?d?d?d  
  
# 8-character lowercase  
hashcat -m 0 -a 3 hashes.txt ?l?l?l?l?l?l?l?l
```

3. **Reflection:** When are mask attacks most effective?

4.5 Exercise 5: GPU Acceleration

Exercise 5: GPU Acceleration

Objective: Learn how to use GPU acceleration for faster cracking.

1. Check available devices:

```
hashcat -I
```

2. Run with GPU:

```
hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt -  
0 -w 3
```

3. Benchmark:

```
hashcat -b -m 0  
hashcat -b -m 100  
hashcat -b -m 1400
```

4. **Reflection:** Why is GPU acceleration so powerful?

4.6 Exercise 6: Rainbow Tables

Exercise 6: Rainbow Tables

Objective: Understand how rainbow tables work.

1. Install tools:

```
sudo apt-get install rcracki-mt
```

2. Generate a small table:

```
rtgen md5 loweralpha 1 5 0 1000 1000 0
```

3. Sort and use:

```
rtsort .  
rcrack . -h 482c811da5d5b4bc6d497ffa98491e38
```

4. **Reflection:** What are the limitations of rainbow tables?

5 Defensive Measures

Defensive Recommendations

1. Strong Password Policies:

- Minimum 12 characters
- Complexity requirements
- Password history (prevent reuse)
- Maximum age (90 days)

2. Account Protection:

- Multi-factor authentication (MFA)
- Account lockout after failed attempts
- Monitor for compromised credentials

3. Technical Controls:

- Use strong hashing (bcrypt, scrypt, Argon2)
- Implement salting and key stretching

6 Deliverables

Submit a report containing:

1. Hash values generated in Exercise 1
2. Cracking results from dictionary attacks
3. Rule-based cracking results
4. Mask attack results
5. Performance benchmarks
6. Screenshots of successful password recovery
7. A reflection on password security
8. Recommendations for a strong password policy

7 Additional Resources

- **Hashcat Documentation:** <https://hashcat.net/wiki/>
- **John the Ripper Documentation:** <https://www.openwall.com/john/>
- **OWASP Password Storage:** https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html